

Documentação das Telas

Simulador de Escalonamento de SO Multitarefa

Descrição de cada elemento visual apresentado pelo programa e da origem (estrutura de dados / arquivo-fonte / linha do arquivo de configuração) de cada informação exibida.

Caso de teste analisado: **caso-teste-mc-010-priopenv.txt** (algoritmo PRIOPEnv — Prioridade Preemptiva com Envelhecimento, 2 CPUs, 10 tarefas)

1. Como as telas são geradas

O programa produz três grupos de telas, todas alimentadas pelas mesmas estruturas de dados internas (Task, CPU, GanttEntry), que por sua vez nascem da leitura do arquivo .txt de entrada:

Grupo de telas	Onde é gerada	Quando aparece
Telas de texto no terminal (menu, configuração, resumo de tarefas, Gantt em ASCII/ANSI)	main.c e gantt.c (função gantt_print_terminal)	Durante toda a execução, no console
Gráfico de Gantt em SVG (arquivo gantt.svg)	gantt.c, função gantt_save_svg	Ao final da simulação completa, ou a cada passo no modo passo-a-passo

Fluxo de dados resumido: o arquivo .txt é lido por load_config() (config.c), que preenche o vetor de Task (task.h) e a lista de eventos de cada tarefa (parse_events, events.c). A cada tick, simulation_step() (simulation.c) atualiza o estado de tarefas e CPUs chamando o escalonador (scheduler.c) e depois grava uma "fotografia" daquele tick em GanttHistory através de gantt_record(). É esse histórico que gantt.c percorre para desenhar tanto o Gantt do terminal quanto o SVG.

2. Telas em modo texto (terminal)

2.1 — Banner e configuração carregada

```
=====
Simulador de SO Multitarefa - v0.5B
=====

Configuracoes carregadas de 'caso-teste-mc-010-priopenv.txt':
Algoritmo : PRIOPENV
Quantum   : 3
CPUs      : 2
Alpha     : 1 (envelhecimento)
```

Origem dos dados: impressa por `main.c` logo após `load_config()`. Os quatro valores vêm diretamente da **primeira linha** do `.txt` ("`PRIOPENV;3;2;1`"), que `load_config()` (`config.c`) separa por `'` na ordem algoritmo, quantum, número de CPUs e alpha, preenchendo a `struct Config`.

2.2 — Menu de seleção de algoritmo

```
Selecione o algoritmo (ENTER para manter PRIOPENV):
1 - FCFS (First-Come, First-Served)      [cooperativo]
2 - RR (Round Robin)                     [preemptivo]
3 - SJF (Shortest Job First)              [cooperativo]
4 - SRTF (Shortest Remaining Time First) [preemptivo]
5 - PRI0c (Prioridade Cooperativo)        [cooperativo]
6 - PRI0p (Prioridade Preemptivo)         [preemptivo]
7 - PRI0d (Prioridade Dinamica/Envelhec.) [preemptivo]
Opcao:
```

Origem dos dados: texto fixo em `main.c`. A escolha do usuário sobrescreve `config.algorithm`, que é o valor lido por `schedule()` (`scheduler.c`) a cada tick para decidir qual critério de escalonamento usar.

2.3 — Lista de tarefas carregadas

```
Tarefas carregadas (10):
T1  | Cor: #0000FF | Ingresso: 3 | Duracao: 9 | Prio: 1 | Eventos: 2
T2  | Cor: #00FF00 | Ingresso: 3 | Duracao: 10 | Prio: 6 | Eventos: 0
T3  | Cor: #FF0000 | Ingresso: 1 | Duracao: 30 | Prio: 4 | Eventos: 4
...
```

Origem dos dados: laço em `main.c` que imprime, para cada `Task`, os campos `id`, `color`, `arrival_time`, `duration`, `priority`, `event_list.count`. Cada um vem de uma coluna da linha da tarefa no `.txt` (ex.: "`t01;0000FF;3;9;1;ML01:00;MU01:05;`"), conforme o parsing manual por `'` em `load_config()` (`config.c`): campo 1 = `id`, campo 2 = `cor`, campo 3 = `ingresso`, campo 4 = `duração`, campo 5 = `prioridade`, e o restante da linha = `eventos` (passado inteiro para `parse_events()`, `events.c`).

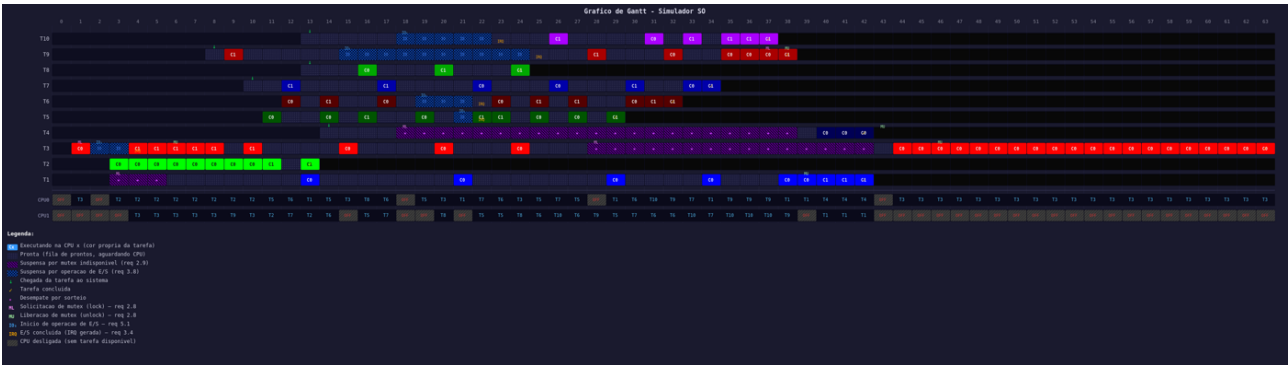
2.4 — Gráfico de Gantt no terminal (ANSI)

No modo texto, `gantt_print_terminal()` (`gantt.c`) desenha uma versão colorida com códigos de escape ANSI, com exatamente os mesmos estados e ícones descritos em detalhe na seção 3 (a versão gráfica em SVG). A diferença é só de renderização: no terminal, "executando" vira um bloco de cor de fundo RGB com `Cx`, "pronta" vira "■■■■■" cinza, "suspensa por mutex" vira fundo roxo com ■, "suspensa por E/S" vira fundo azul com "IO", e chegada/fim aparecem como `↓t=N` / `✓t=N` ao lado do nome da tarefa.

Origem dos dados: a mesmíssima `struct GanttHistory` usada para gerar o SVG — ou seja, tudo o que está na seção 4 desta documentação vale igualmente para esta tela.

3. Tela gráfica — Gráfico de Gantt (gantt.svg)

Esta é a tela principal do sistema: um arquivo SVG gerado por `gantt_save_svg()` ao final da simulação (ou a cada passo, no modo interativo). Abaixo, a visão geral do arquivo analisado (64 ticks, 10 tarefas, 2 CPUs), seguida do detalhamento de cada elemento.

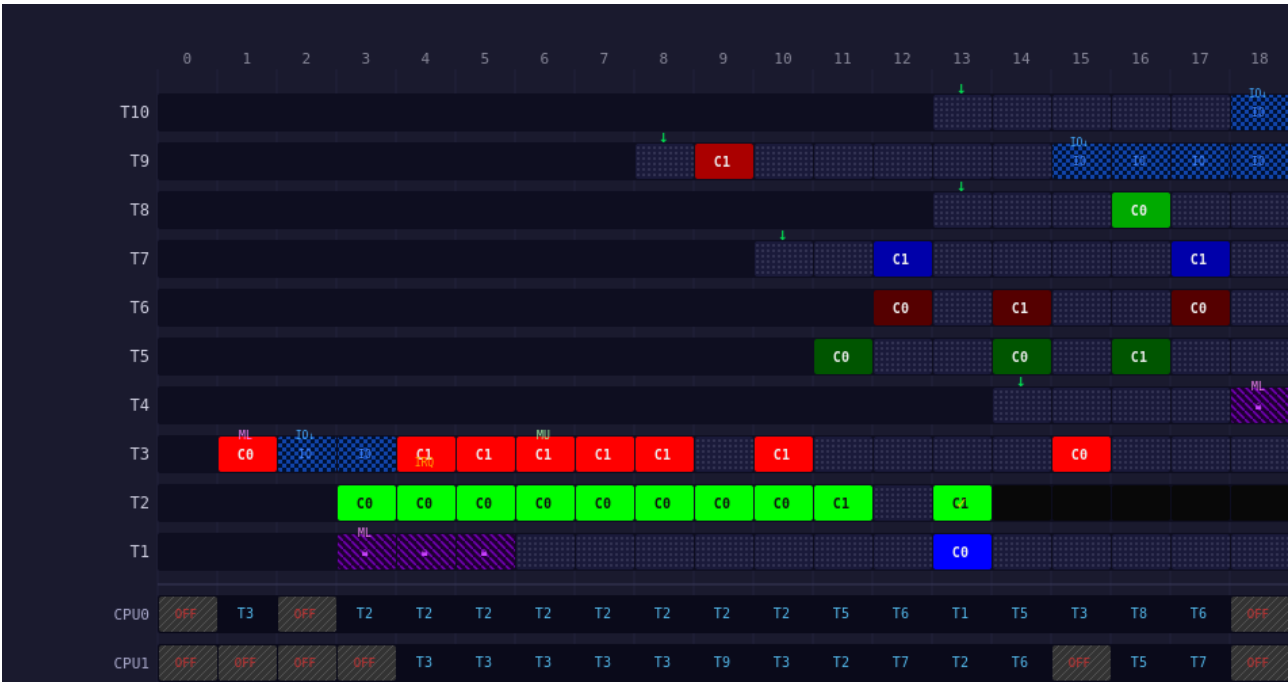


Visão geral do `gantt.svg` gerado para o caso de teste (imagem reduzida; os detalhes de cada elemento são ampliados a seguir).

3.1 — Cabeçalho e eixo do tempo

O título "Gráfico de Gantt - Simulador SO" é texto fixo escrito por `gantt_save_svg()`. Os números do eixo horizontal (0, 1, 2, ...) vêm de `history->entries[t].tick`: cada posição do histórico corresponde a um tick de `sim->clock` (simulation.c), incrementado uma vez por chamada de `simulation_step()`. As linhas verticais finas são apenas guias visuais entre colunas.

3.2 — Linhas de tarefas (uma por tarefa do .txt)



Recorte real do `gantt.svg` gerado: colunas 0 a 18, tarefas T1 a T10. Mostra os principais estados de célula e ícones lado a lado.

Elemento visual	Significado	De onde vem o dado
Rótulo "TX" à esquerda da linha	Identificador da tarefa	<code>tasks[i].id</code> — campo 1 da linha da tarefa no <code>.txt</code> (ex.: "t01" → 1)

Bloco colorido com "C0"/"C1"	Tarefa executando naquele tick, na CPU indicada	Cor: <code>tasks[i].color</code> (campo 2 do .txt). CPU: <code>cpus[c].task_id/active</code> , definidos por <code>assign_tasks()</code> (scheduler.c) e consumidos por <code>execute_tick()</code> (simulation.c)
Célula pontilhada (fundo azul-escuro)	Tarefa pronta , aguardando CPU livre	<code>tasks[i].state == READY</code> , definido pelo escalonador quando a tarefa não foi escolhida naquele tick
Hachura roxa diagonal + ícone de cadeado	Tarefa suspensa por mutex indisponível	<code>state == SUSP_MUTEX</code> , definido em <code>check_events()</code> (simulation.c) quando <code>mutex_lock()</code> retorna "bloqueado"; o identificador do mutex vem do evento "MLxx:tt" no .txt (events.c)
Padrão quadriculado azul + texto "IO"	Tarefa suspensa por operação de E/S	<code>state == SUSP_IO</code> , definido em <code>check_events()</code> ao processar o evento "IO:tt-dd" do .txt; a largura do bloco (nº de ticks) é a duração dd do evento
Célula preta lisa	Tick em que a tarefa já está concluída e não voltará a rodar	<code>state == FINISHED</code> , mas o tick atual não é o tick de conclusão (não há mais nada a desenhar sobre ele)
Seta verde ↓ acima de uma célula	Tick em que a tarefa chegou ao sistema	<code>task_arrived[i]</code> , calculado em <code>gantt_record()</code> como <code>state==READY && arrival_time==tick</code> ; <code>arrival_time</code> é o campo 3 do .txt
Símbolo ✓ amarelo sobre a última célula colorida	Tick em que a tarefa terminou sua execução	<code>task_finished[i] = state==FINISHED && finish_time==tick</code> . <code>finish_time</code> é gravado em <code>execute_tick()</code> quando <code>remaining_time</code> chega a zero. Aparece sobre a célula colorida (e não numa célula preta separada) porque esse tick foi, de fato, um tick real de execução
Estrela magenta ★ (não observada neste teste)	Indica que houve desempate por sorteio naquele tick	<code>lottery_tick</code> , retornado por <code>assign_tasks()</code> quando <code>tiebreak()</code> (scheduler.c) precisa cair em <code>rand()%2</code> por empate total entre duas tarefas
Tag "ML" no topo da célula	A tarefa solicitou um lock de mutex naquele tick	<code>task_mutex_lock[i]</code> , ligado quando o evento "MLxx:tt" do .txt dispara em <code>check_events()</code>
Tag "MU" no topo da célula	A tarefa liberou um mutex naquele tick	<code>task_mutex_unlock[i]</code> , ligado pelo evento "MUxx:tt" do .txt
Tag "IO↓" no topo da célula	Início de uma operação de E/S	<code>task_io_start[i]</code> , ligado pelo evento "IO:tt-dd" do .txt
Tag "IRQ" na base da célula	A E/S terminou naquele tick (interrupção)	<code>task_irq[i]</code> , vem de <code>check_irq()</code> (simulation.c), que consulta a fila de E/S (<code>io_queue</code>) agendada com <code>clock + duração</code> no momento do "IO↓"

3.3 — Exemplos ampliados

Suspensão por mutex (T1, ticks 3-5):



T1 tenta travar o mutex 1 assim que começa a rodar ("ML01:00" no .txt = lock no instante relativo 0). Como o mutex já está em uso por outra tarefa, T1 fica SUSP_MUTEX por 3 ticks até ser liberado.

Suspensão por E/S (T9, ticks 9-18):



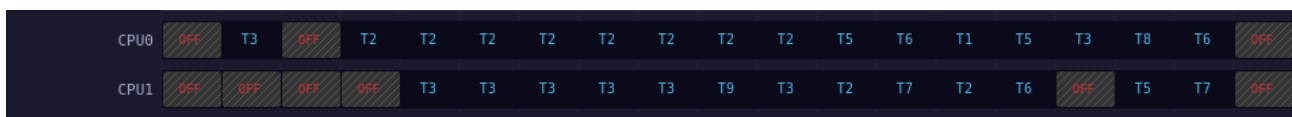
T9 dispara "IO:01-10" (E/S de duração 10) logo após seu primeiro tick de execução; o bloco quadriculado azul tem exatamente 10 células de largura, e a tag "IRQ" marca o tick seguinte, em que a E/S termina e a tarefa volta a ficar pronta.

Conclusão sobreposta à execução (T2, ticks 11-14):



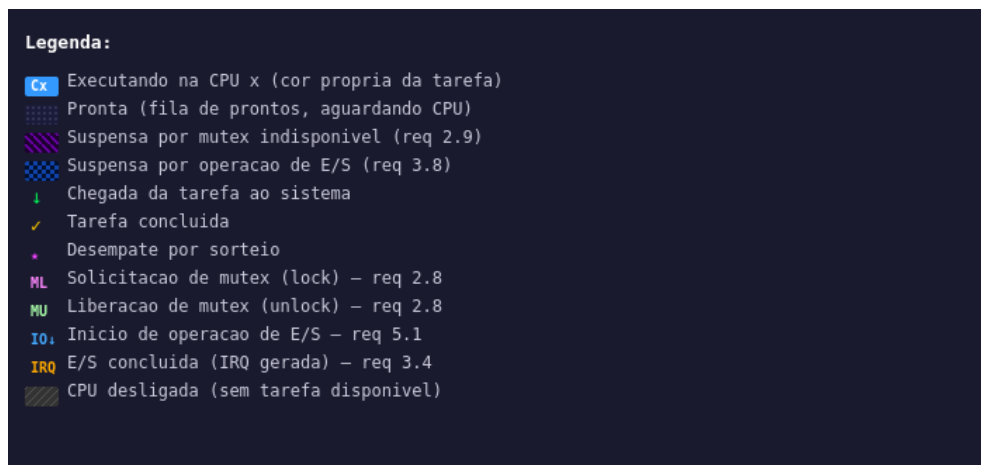
T2 é preemptada por um tick (célula pontilhada), retoma a CPU e conclui sua última unidade de execução — o ✓ aparece sobre essa célula colorida, e somente a partir do tick seguinte a linha passa a mostrar células pretas.

3.4 — Linhas de CPU (CPU0 / CPU1)



Uma linha por CPU configurada (config.cpu_count, campo 3 da primeira linha do .txt). Em cada tick mostra o rótulo da tarefa em execução ("TX") ou, quando não há nenhuma tarefa disponível, uma célula hachurada com "OFF". Os dados vêm de cpus[c].task_id e cpus[c].active (scheduler.c/simulation.c), gravados por tick em e->cpu_task[c] / e->cpu_active[c] dentro de gantt_record().

3.5 — Legenda



Bloco de texto e amostras de padrão escritos uma única vez, ao final do arquivo, por gantt_save_svg(). Não depende dos dados da simulação — é a mesma legenda em qualquer caso de teste.

4. Tabela-resumo: do arquivo .txt até a tela

Rastreabilidade completa: cada dado exibido em qualquer uma das telas tem origem em uma coluna do arquivo de configuração, passa por uma struct em memória, e é lido por uma função de desenho.

Campo no .txt	Struct / campo em memória	Onde é lido para desenhar
Linha 1, campo 1 (algoritmo, ex. "PRIOPEnv")	Config.algorithm	schedule() (scheduler.c) — decide o critério de escolha a cada tick
Linha 1, campo 2 (quantum)	Config.quantum	assign_tasks() — preempção por esgotamento de fatia de tempo
Linha 1, campo 3 (nº de CPUs)	Config.cpu_count	Nº de linhas "CPUx" desenhadas no Gantt
Linha 1, campo 4 (alpha)	Config.alpha	assign_tasks() — incremento de priority_dynamic por tick em fila (envelhecimento)
Tarefa, campo 1 (id, ex. "t01")	Task.id	Rótulo "TX" de cada linha do Gantt
Tarefa, campo 2 (cor hex)	Task.color	Cor de fundo das células "executando"
Tarefa, campo 3 (ingresso)	Task.arrival_time	Coluna onde a linha da tarefa começa; ícone de chegada ↓
Tarefa, campo 4 (duração)	Task.duration → remaining_time	Nº total de células "executando" até a tarefa concluir
Tarefa, campo 5 (prioridade)	Task.priority / priority_dynamic	Usada por schedule_prio_* para decidir quem ocupa a CPU (não aparece como texto na tela, mas define a ordem das células coloridas)
Tarefa, eventos "MLxx:tt"	Event{type=EVT_MUTEX_LOCK, . . .}	Tag "ML" + célula hachurada roxa (se bloquear)
Tarefa, eventos "MUxx:tt"	Event{type=EVT_MUTEX_UNLOCK, . . .}	Tag "MU"
Tarefa, eventos "IO:tt-dd"	Event{type=EVT_IO, . . .}	Tag "IO↓" + bloco quadriculado azul de largura dd + tag "IRQ" ao final

Documento gerado a partir da leitura direta do código-fonte (main.c, config.c, events.h/events.c, task.h, scheduler.c, simulation.c, gantt.c) e do arquivo gantt.svg produzido para o caso de teste caso-teste-mc-010-priopenv.txt, após a correção dos bugs de temporização de eventos, loop infinito e do tick final "comido" no Gantt.